

langchain-ai/langgraph

LangGraph : une suite de tests massive et une CI stricte, mais aucune couverture mesurée et 528 tickets ouverts

Build resilient agents.

66

FICHIERS DE TEST
pytest

2

WORKFLOWS CI

81

BUGS ÉTIQUETÉS

528

ISSUES OUVERTES

☑ Que fait ce projet et quels sont ses parcours critiques ?

VERDICT

Les parcours critiques ne sont pas des fonctionnalités mais des scénarios de panne — reprise après crash, interruption, reprise de mémoire — parce que c'est ce que le projet promet.

LangGraph est un cadre d'orchestration bas niveau pour agents à état : des programmes pilotés par un modèle de langage qui tournent longtemps, gardent leur état et reprennent après une panne.

Le projet met en avant quatre promesses, et ce sont elles qu'un plan de test doit attaquer en premier :

- ❑ **exécution durable** : l'agent reprend « exactement là où il s'est arrêté » après un échec ;
- ❑ **humain dans la boucle** : on inspecte et on modifie l'état à n'importe quel point de l'exécution ;
- ❑ **mémoire** : mémoire de travail à court terme et mémoire persistante entre sessions ;
- ❑ **déploiement en production** : charges longues et à état.

Chacune de ces quatre promesses ne se vérifie pas par un cas nominal mais par une panne provoquée : couper au milieu d'une étape, reprendre, comparer l'état. C'est la nature même du produit qui impose des cas de test destructifs.

Le périmètre est large : 41 384 étoiles, 6 990 forks, licence MIT, dernier envoi de code le 2026-09-09.

Comment est-il testé aujourd'hui ?

VERDICT

97 modules de test recensés, mais 6 fichiers concentrent le gros du volume sur le seul moteur Pregel — c'est là que porte l'effort amont, et donc là que votre propre suite est la moins utile.

- `.github/` 17 workflows GitHub Actions, scripts CI
- `docs/` documentation
- `examples/` exemples d'usage, notebooks
- `libs/` code principal : 9 paquets, 8 en Python, sdk-js réduit à un README
- `.gitignore` exclusions git
- `.markdownlint.json` config lint Markdown
- `AGENTS.md` consignes pour les agents IA
- `CLAUDE.md` consignes pour Claude Code
- `LICENSE` licence MIT
- `Makefile` pilote du monorepo : install, lint, test via uv
- `README.md` présentation du projet

Monorepo : neuf paquets sous `libs/` , aucun dossier de tests à la racine, chaque paquet a le sien. Le cadre est **pytest**, le dossier principal est `libs/langgraph/tests` avec 66 entrées listées.

Au total, **97 modules `test_*.py` ** ont été dénombrés dans les sept dossiers énumérés :

PAQUET	MODULES	TEST_*.PY	CE QU'IL COUVRE
libs/langgraph	51		moteur Pregel, streaming, reprise, canaux
libs/sdk-py	15		client HTTP Python
libs/prebuilt	10		agent ReAct, ToolNode, ValidationNode
libs/checkpoint-sqlite	8		sauvegarde d'état SQLite
libs/checkpoint	6		interface, sérialisation, chiffrement
libs/checkpoint-postgres	6		sauvegarde d'état Postgres
libs/checkpoint-conformance	1		contrat commun aux trois checkpointers

Le volume est très concentré : `test_pregel.py` et `test_pregel_async.py` pèsent 619 Ko à eux deux, et six fichiers portent l'essentiel de la suite du paquet principal. Le moteur d'exécution est testé massivement et de façon centralisée, pas éclaté par module.

Deux points favorables au testeur : une suite de conformité partagée (`test_conformance_delta.py` , identique dans les trois implémentations de checkpointer) et des tests par instantané avec syrupy, plus Postgres et Redis montés en conteneurs pour les tests.

☑ Que dit la CI et quand tourne-t-elle ?

VERDICT

La CI garantit que les tests passent sur 5 versions de Python et ne salissent rien, mais elle ne dit jamais quelle part du code est testée — la couverture réelle est inconnue et non surveillée.

Deux workflows relevés : CI (`.github/workflows/ci.yml`), déclenché à la main, sur chaque envoi vers `main` et sur chaque proposition de changement ; et `test` (`.github/workflows/_test_langgraph.yml`), appelé par le premier. Il existe 17 workflows en tout, dont un seul a été lu en entier.

Ce que la CI fait vraiment, dans l'ordre :

- 1 `changes` filtre par chemins modifiés — un changement hors `libs/` ne déclenche aucun test ;
- 2 `lint` sur une matrice de 8 paquets, `test` sur 7 d'entre eux, `test-langgraph` à part pour le cœur ;
- 3 `check-sdk-methods` et `check-schema` échouent si le schéma généré de la CLI change sans être commité ;
- 4 tests d'intégration CLI et SDK Python, puis `ci_success` qui agrège tout.

Le workflow du cœur est exigeant : matrice **Python 3.10 à 3.14**, dépendances gelées (`uv sync --frozen`), exécution parallèle via `make test_parallel` , une seconde passe en sérialisation msgpack stricte sur Python 3.13 pour les deux gros fichiers Pregel, et une dernière étape qui échoue si la suite laisse le moindre fichier derrière elle.

En local, la commande est `make test` , et `TEST=path/to/test.py make test` pour un seul fichier ; l'installation utilisateur est `pip install -U langgraph` .

Le trou est net : **aucune mesure de couverture**. `pytest-cov` est installé mais ni `ci.yml` ni `_test_langgraph.yml` ne produit de rapport ni n'impose de seuil.

📌 Où sont les bugs connus ?

VERDICT

528 ouverts contre 7 refermés en un mois, et les cinq tickets les plus discutés portent tous sur la reprise après panne — c'est le point de fragilité à couvrir en premier.



528 tickets ouverts, 7 refermés sur les trente derniers jours. Parmi les tickets relevés, 81 portent l'étiquette `bug`, 297 l'étiquette `external` et 3 `internal`. Le rapport entre ce qui entre et ce qui se referme est le premier signal de risque du projet.

Les tickets les plus discutés dessinent un thème unique — la reprise après panne et la persistance d'état :

#	SUJET	COMMENTAIRES
7417	appels d'outil longs (~180 s) rejoués silencieusement depuis un checkpoint	51
8039	<code>durability="sync"</code> : ordre d'écriture non garanti, reprise dépendante de l'hôte	36
7714	sérialisation : 85 % de volume de stockage en trop, 37,8 % de jetons en trop	21
8764	crash avant le premier checkpoint durable : exécution perdue sans trace	20
7780	<code>interrupt()</code> dans une boucle provoque des reprises en trop	19

Ces cinq numéros sont des cas de test à écrire tels quels : chacun décrit une reproduction et un comportement attendu qui diverge.

Donnée non relevée : aucun ticket « good first issue » n'apparaît, la liste est vide ; à vérifier dans <https://github.com/langchain-ai/langgraph/issues>.

☑ Comment reproduire et signaler un bug ici ?

VERDICT

Un bug non accompagné d'un exemple copiable et de `python -m langchain_core.sys_info` ne sera pas traité, et une correction externe sans ticket approuvé est fermée d'office.

Il n'existe **pas de fichier** `CONTRIBUTING.md` dans le dépôt : ni à la racine, ni dans `.github/`, ni dans `docs/`. Les règles complètes sont hébergées hors du dépôt¹.

Ce que le dépôt impose, lui, est dans ses gabarits, et c'est une liste de contrôle stricte :

- 1 pas d'issue vierge possible (`blank_issues_enabled: false`) — une question part au forum, pas dans les tickets ;
- 2 sept cases à cocher obligatoires, dont « ce n'est pas une question d'usage » et « ce n'est pas résolu par une montée en dernière version stable » ;
- 3 un **exemple minimal reproductible qu'un mainteneur peut copier-coller et lancer tel quel** — capture d'écran refusée ;
- 4 la sortie de `python -m langchain_core.sys_info`, obligatoire ;
- 5 message d'erreur et pile d'appels complets.

Pour une correction, le gabarit de proposition de changement exige un titre `TYPE(SCOPE): DESCRIPTION`, une ligne `Fixes #xx` obligatoire pour toute contribution externe, et `make format`, `make lint`, `make test` verts. Une proposition externe sans ticket approuvé au préalable est fermée.

Le débit donne l'ordre de grandeur de l'attente : **245 propositions ouvertes**, 33 intégrées sur trente jours, dont une large majorité de montées de version automatiques plutôt que de correctifs.

☑️ Quelles zones sont peu couvertes ?

VERDICT

La seule zone d'ombre qui compte est la même partout — la couverture n'est mesurée nulle part, donc tout jugement sur « ce qui est testé » repose sur la lecture des noms de fichiers.

L'activité est régulière mais mince : entre 2 et 25 commits par semaine sur les douze dernières, dernier commit le 2026-09-09, et un bus factor de 4 sur dix contributeurs relevés.

Les zones sans filet, telles que relevées :

ZONE	CE QUI MANQUE	CONSÉQUENCE POUR LE TEST
tout le code	aucun seuil ni rapport de couverture en intégration continue	impossible de savoir ce qui n'est pas testé sans le mesurer soi-même
libs/sdk-js	le dossier ne contient qu'un README.md	aucun test JavaScript ici : le SDK JS se teste dans un autre dépôt
libs/checkpoint-conformance	un seul module de test	le contrat commun aux trois checkpointers tient sur un fichier
signalement de faille	pas de SECURITY.md , aucun canal déclaré	un bug de sécurité n'a pas de voie de remontée privée

Donnée non relevée : le contenu de libs/cli/tests/unit_tests et libs/cli/tests/integration_tests n'a pas été énuméré, non plus que les sous-dossiers integration/ et streaming/ de libs/sdk-py/tests ; à vérifier dans <https://github.com/langchain-ai/langgraph/tree/main/libs/cli/tests>.



Plan de test en 5 points

- 1 **Mesurer avant de juger.** Lancer la suite avec couverture (`pytest-cov` est déjà installé) sur `libs/langgraph` et publier le chiffre : c'est la donnée qui manque au projet comme à vous.
- 2 **Rejouer les cinq tickets chauds comme cas de test.** #7417, #8039, #7714, #8764 et #7780 décrivent chacun une reproduction et un écart de comportement sur la reprise après panne.
- 3 **Tester la panne, pas le nominal.** Couper l'exécution avant le premier checkpoint durable, pendant un appel d'outil long, et dans une boucle `interrupt()` : les trois scénarios que le projet promet de tenir et que ses tickets contestent.
- 4 **Reproduire la matrice amont sur votre cible.** La CI valide Python 3.10 à 3.14 en dépendances gelées ; ne validez que la version que vous déployez, mais avec le même `uv sync --frozen`, sinon vous testez autre chose.
- 5 **Vérifier les trois checkpointers contre le même contrat.** Mémoire, SQLite et Postgres partagent `test_conformance_delta.py` : si vous en écrivez un quatrième ou changez de backend, c'est ce fichier qui fait foi.

RÉFÉRENCES PUBLIQUES

☑ Sources

- 1 <https://docs.langchain.com/oss/python/contributing/overview>